

Lab 03

git continued, local branches, reading quiz

Today's plan

01 **Git commands continued**

Review commands, git rm, git mv

02 **Intro to text editors**

vim, emacs, nano

03 **Git branches**

Git checkout, git branch, git switch

04 **Merging**

Merging branches

05 **Reading Checkpoint # 2**

Reading: Guide for Reproducible Research

Announcements

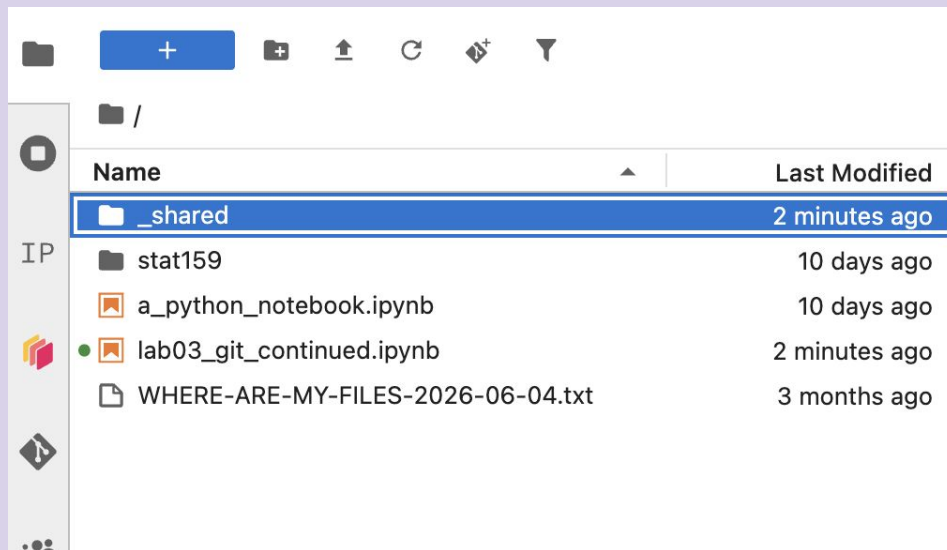
Shared folder on datahub

The datahub has been configured to have a shared folder you all have access to, where we will put:

- Lab notebooks
- Shared datasets
- Github authentication notebook

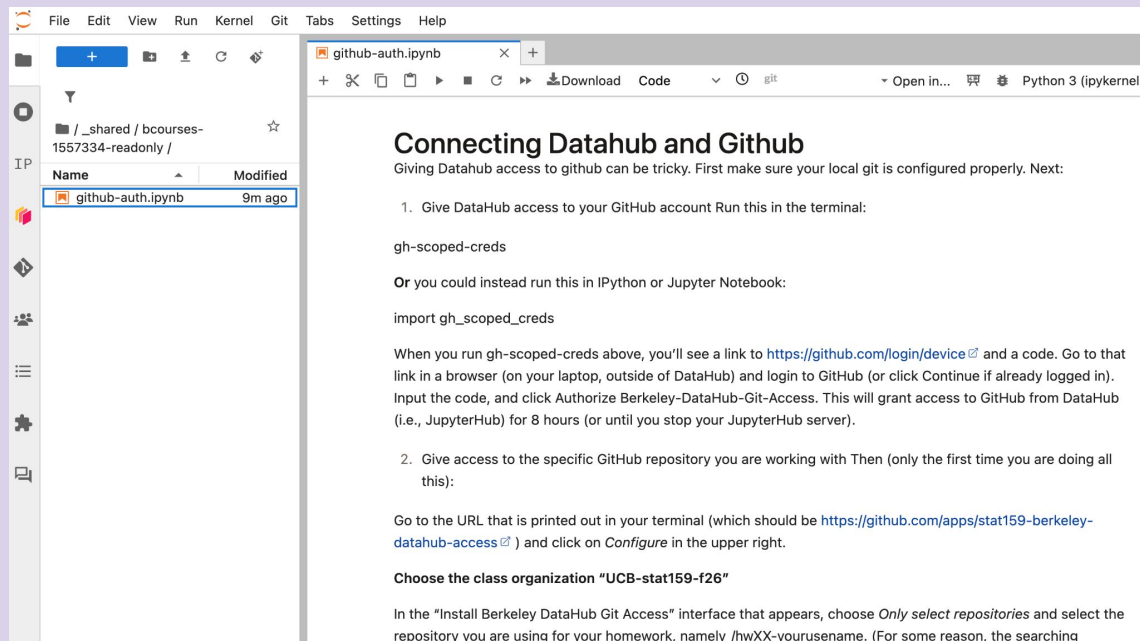
When using files from here:

- Copy from the shared folder into your own folder



Github authentication notebook on datahub

Under the shared folder, we have put a github authentication notebook. Please copy it into your stat159 folder for future reference.



The screenshot shows a Jupyter Notebook interface. On the left, a file explorer displays the directory structure: `/_shared/bcourses-1557334-readonly/`. Below this, a table lists files:

Name	Modified
github-auth.ipynb	9m ago

The main notebook area is titled "github-auth.ipynb" and contains the following text:

Connecting Datahub and Github

Giving Datahub access to github can be tricky. First make sure your local git is configured properly. Next:

1. Give DataHub access to your GitHub account Run this in the terminal:
`gh-scoped-creds`
Or you could instead run this in IPython or Jupyter Notebook:
`import gh_scoped_creds`
When you run `gh-scoped-creds` above, you'll see a link to <https://github.com/login/device> and a code. Go to that link in a browser (on your laptop, outside of DataHub) and login to GitHub (or click Continue if already logged in). Input the code, and click Authorize Berkeley-DataHub-Git-Access. This will grant access to GitHub from DataHub (i.e., JupyterHub) for 8 hours (or until you stop your JupyterHub server).
2. Give access to the specific GitHub repository you are working with Then (only the first time you are doing all this):
Go to the URL that is printed out in your terminal (which should be <https://github.com/apps/stat159-berkeley-datahub-access>) and click on *Configure* in the upper right.
Choose the class organization "UCB-stat159-f26"
In the "Install Berkeley DataHub Git Access" interface that appears, choose *Only select repositories* and select the repository you are using for your homework, namely `/hwXX-yourusername`. (For some reason, the searching

Git Commands Continued

Recall from Lab 02

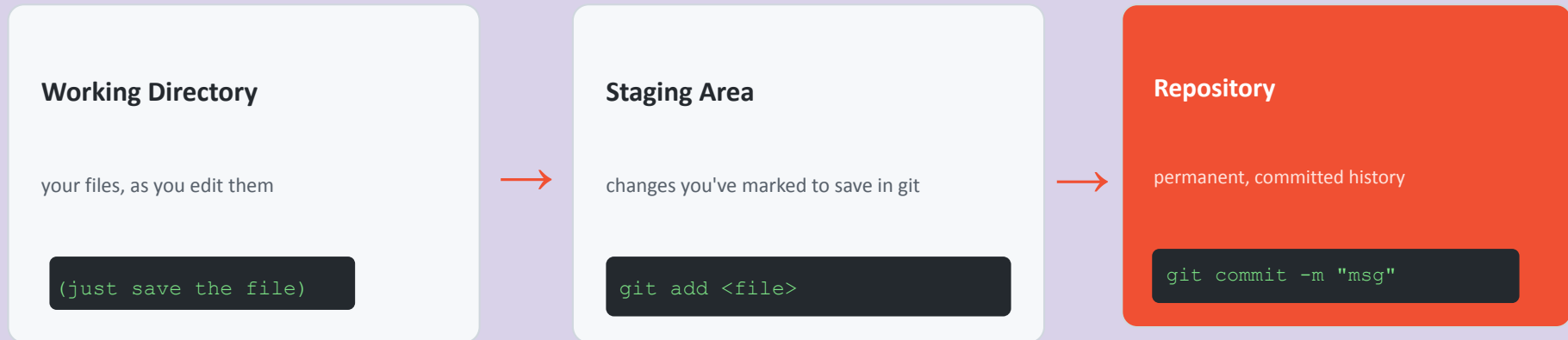
Git is a *version control system* used to track file changes over time

Last week we discussed:

- The structure of git
 - Working directory -> staging area -> repository
- Standard git commands
 - git init
 - git status
 - git add “file.txt”
 - git commit -m “my commit message”
 - git log/git diff

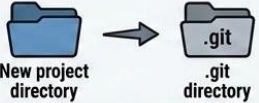
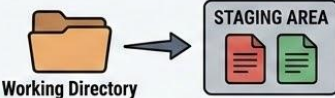
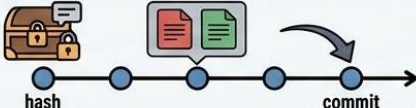
Recall for lab 02: git structure

- Repo (comes from *repository*) = collection of source code
- Working directory vs staging area vs local repo vs remote repo



Recall from lab 02: git commands

GIT ESSENTIALS: COMMON COMMANDS REFERENCE

GIT COMMAND	DESCRIPTION	VISUAL FLOW	EXAMPLE
 git init	Initializes a new Git repository in the current directory, creating the .git folder.		<pre>\$ mkdir my-project \$ cd my-project \$ git init Initialized empty Git repository in /path/to/my-project/.git/</pre>
 git status	Displays the state of the working directory and staging area, showing modified, untracked, and staged files.		<pre>\$ git status On branch main Changes to be committed: (use "git restore --staged <file>..." to unstage) new file: README.md Untracked files: (use "git add <file>..." to include in what will be committed) index.html</pre>
 git log	Shows a chronological commit history of the repository, including commit hashes, authors, dates, and messages.		<pre>\$ git log --oneline a1b2c3d (HEAD -> main) Add initial files e4f5g6h Fix bug #42 j7k8l9m Create README</pre>
 git add	Stages specified files, adding them to the index and preparing them for the next commit.		<pre>\$ git add index.html \$ git add .</pre>
 git commit	Saves the staged snapshot permanently to the repository history with a descriptive message.		<pre>\$ git commit -m "Initial commit" [main (root-commit) a1b2c3d] Initial commit 3 files changed, 25 insertions(+) create mode 100644 index.html create mode 100644 README.md</pre>

`git rm "filename.txt"`

Deletes a tracked file from a git repository

Run inside the project folder with the file to be removed specified

Deletes the file locally and removes it from future commits to the repository

Must run 'git commit' after to solidify changes

Provides the opposite function of git add. If you delete a file locally *without* using git, git will still keep the file in the repo

```
git mv "old_name.txt" "new_name.txt"
```

Moves or renames a previously tracked file

Run inside the project folder with the old and new file names specified

Deletes the old file locally moves it to the new location while retaining the file history

Must run 'git commit' after to solidify changes

Important: if you just rename the file locally, then add it, then remove the old file, git will retain two separate file histories.

git tag

Allows you to label specific commits, such as versions

``git tag v1.0.0'` : tags the current commit as v1.0.0

``git tag -a v1.0.0 -m "Release version 1.0.0"'`: tags the current commit as v1.0.0 with corresponding the message

``git tag -a v1.0.0 <commit-hash>'`: tags the commit at commit-hash as v1.0.0

``git tag -l'` : lists all tags

Important: you must push tags to remote repos specifically with `'git push origin --tags'`

Introduction to text editors

Command line text editors

Sometimes we need to input text in the command line, such as with ‘git commit’, which will prompt a text editor to open in the terminal.

Example text editors include:

- Vi
- Nano
- emacs

Vi is the default git uses, so we will focus on Vi

Command	Vi	Nano	emacs
Exit	:q	Ctrl+X	Ctrl+X Ctrl+C
Save	:w	Ctrl+S	Ctrl+X Ctrl+S

Vi text editor

This can
look scary
the first
few times
you see it!!

[illegible]

Vi text editor

1. Hit “i” to enter insert mode
2. Type your message
3. Hit “esc” button to exit editing mode
4. Type: “:w” to save your edits
5. Type: “:q” to exit the editor

[illegible]

Vi text editor

1. Hit “i” to enter insert mode
2. **Type your message**
3. Hit “esc” button to exit editing mode
4. Type: “:w” to save your edits
5. Type: “:q” to exit the editor

[illegible]

Vi text editor

1. Hit “i” to enter insert mode
2. Type your message
3. **Hit “esc” button to exit editing mode**
4. Type: “:w” to save your edits
5. Type: “:q” to exit the editor

[illegible]

Vi text editor

1. Hit “i” to enter insert mode
2. Type your message
3. Hit “esc” button to exit editing mode
4. **Type: “:w” to save your edits**
5. Type: “:q” to exit the editor

[illegible]

Vi text editor

1. Hit “i” to enter insert mode
2. Type your message
3. Hit “esc” button to exit editing mode
4. Type: “:w” to save your edits
5. **Type: “:q” to exit the editor**

[illegible]

Vi text editor

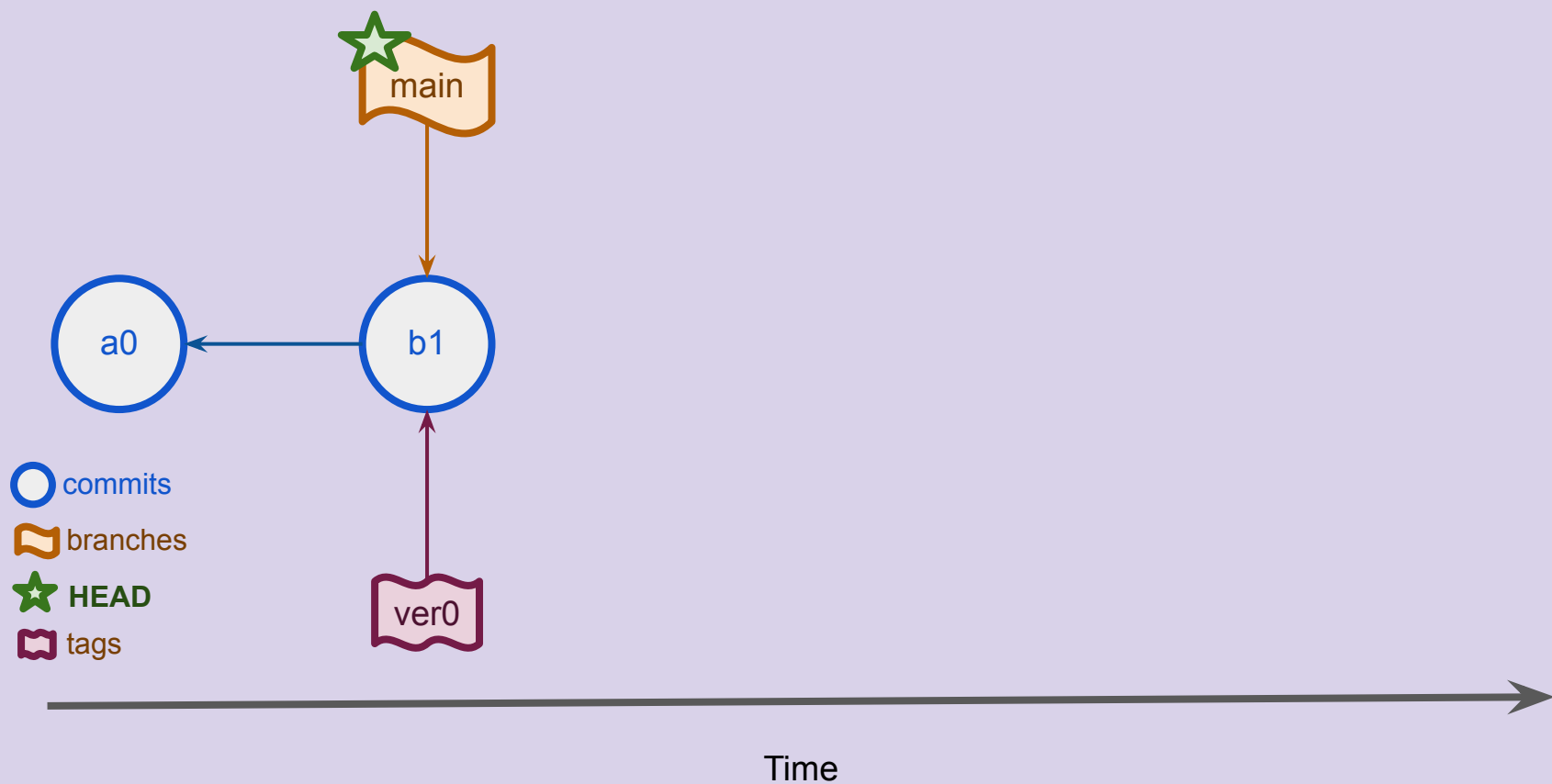
1. Hit “i” to enter insert mode
2. Type your message
3. Hit “esc” button to exit editing mode
4. Type: “:w” to save your edits
5. **Type: “:q” to exit the editor**



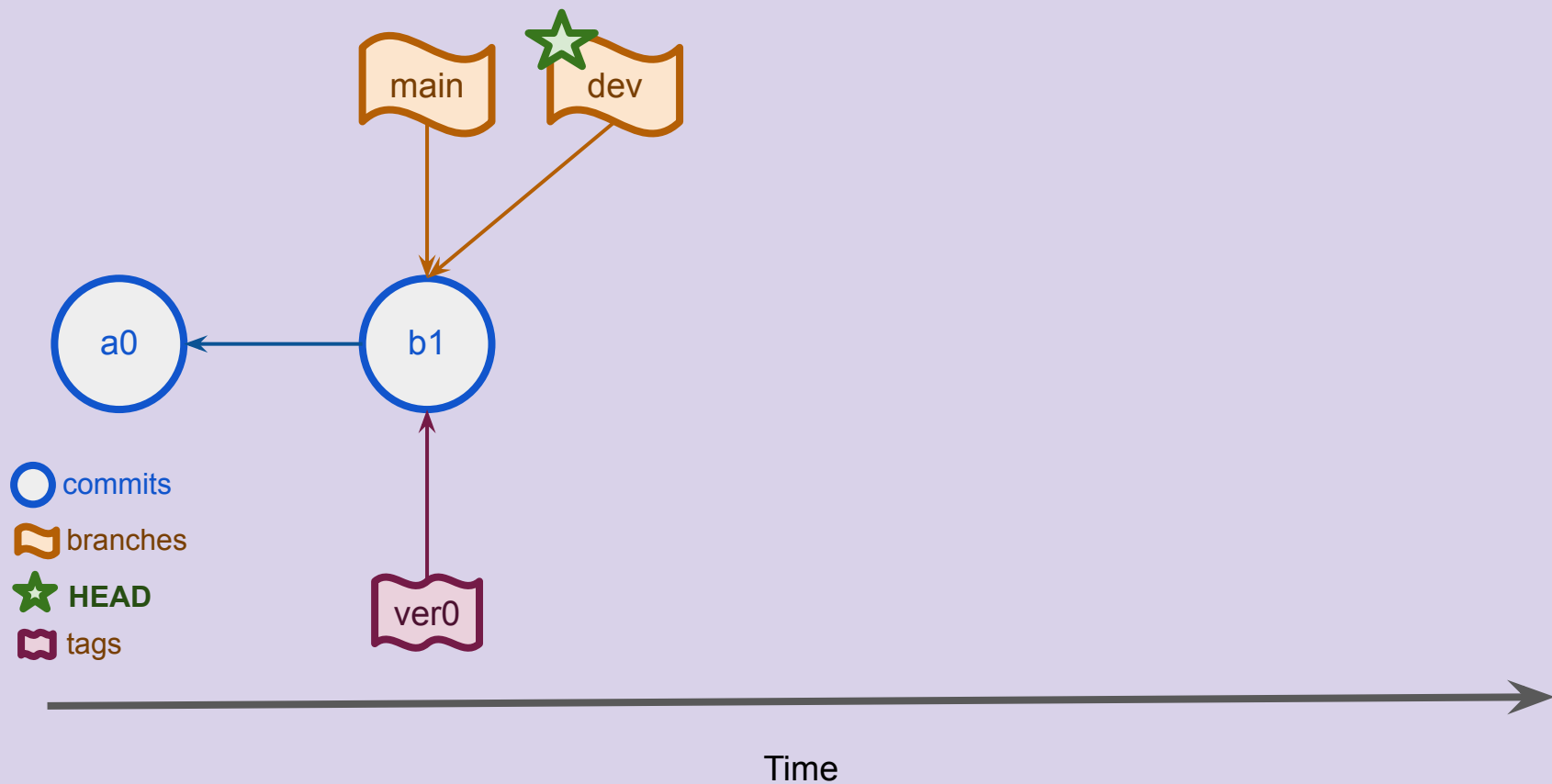
```
Terminal 3 × README.md × lab03_git_continued.ipynb × Launcher ×
jovyan@jupyter-srandrade:~/stat159/labs/lab_practice$ git add README.md
jovyan@jupyter-srandrade:~/stat159/labs/lab_practice$ git commit
[master 36b1800] edit
 1 file changed, 2 insertions(+)
jovyan@jupyter-srandrade:~/stat159/labs/lab_practice$ █
```

Lab Notebook Part 1

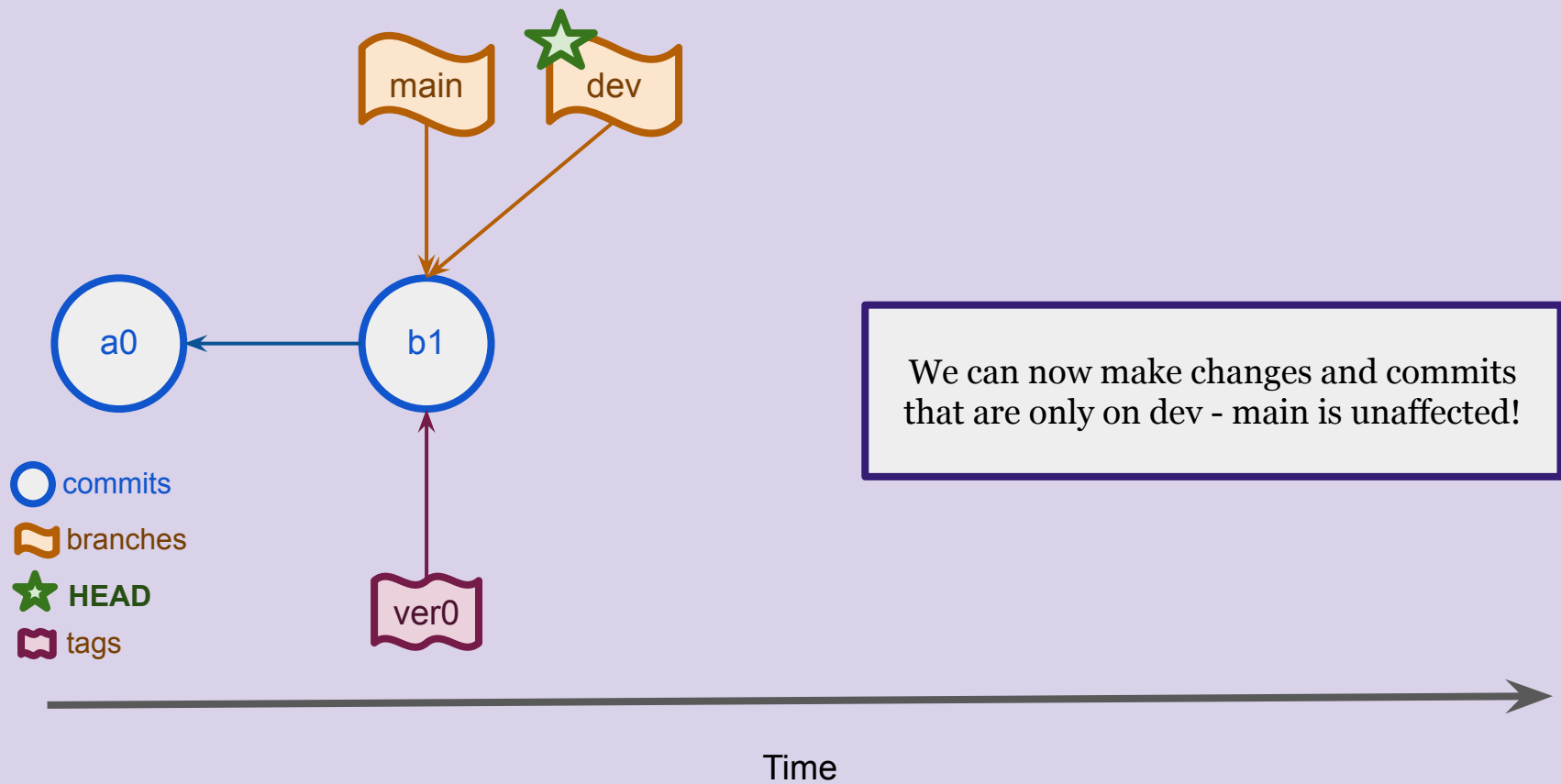
Git Branches



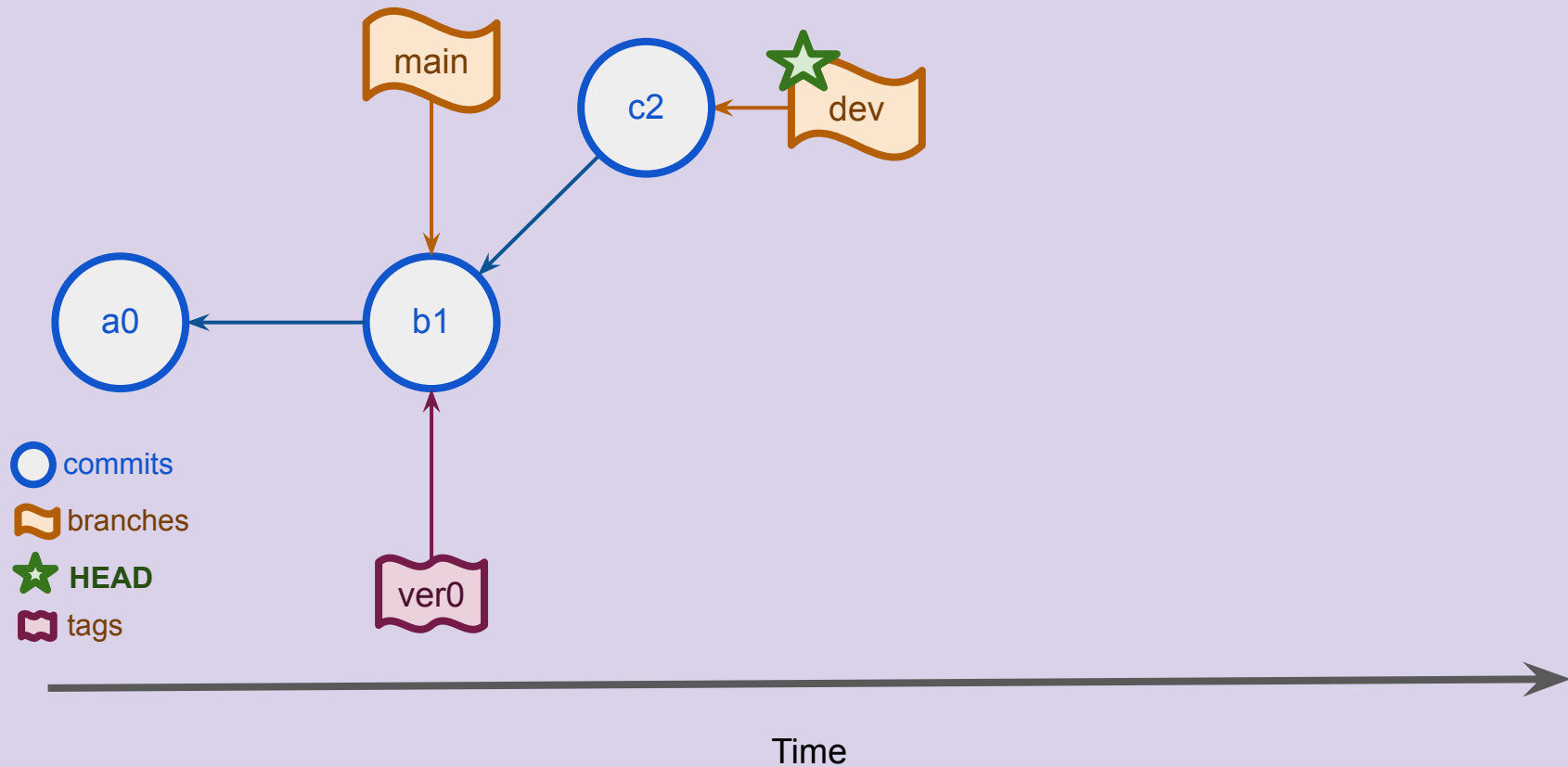
```
$ git tag -a ver0
```



```
$ git switch -c dev
```



```
$ git switch -c dev
```



\$ git commit

git branch

Provides options for branching with different flags

`'git branch'` : lists local branches

`'git branch -r'` : lists remote branches

`'git branch -a'` : lists all local and remote branches

`'git branch branch_name'` : creates a new branch called branch_name

Note you still need to switch to branch_name to make changes on it

git checkout branch_name

Moves head to a different branch/commit and updates files to match

Flag -b creates a new branch “git checkout -b new_branch”

Can also be used to go to specific commit versions or tags e.g.:

“git checkout commit-hash”

“git checkout v0.0.1”

Important: will overwrite all your files to the version of the files on new_branch or commit, be careful if you have non-committed changes

git switch branch_name

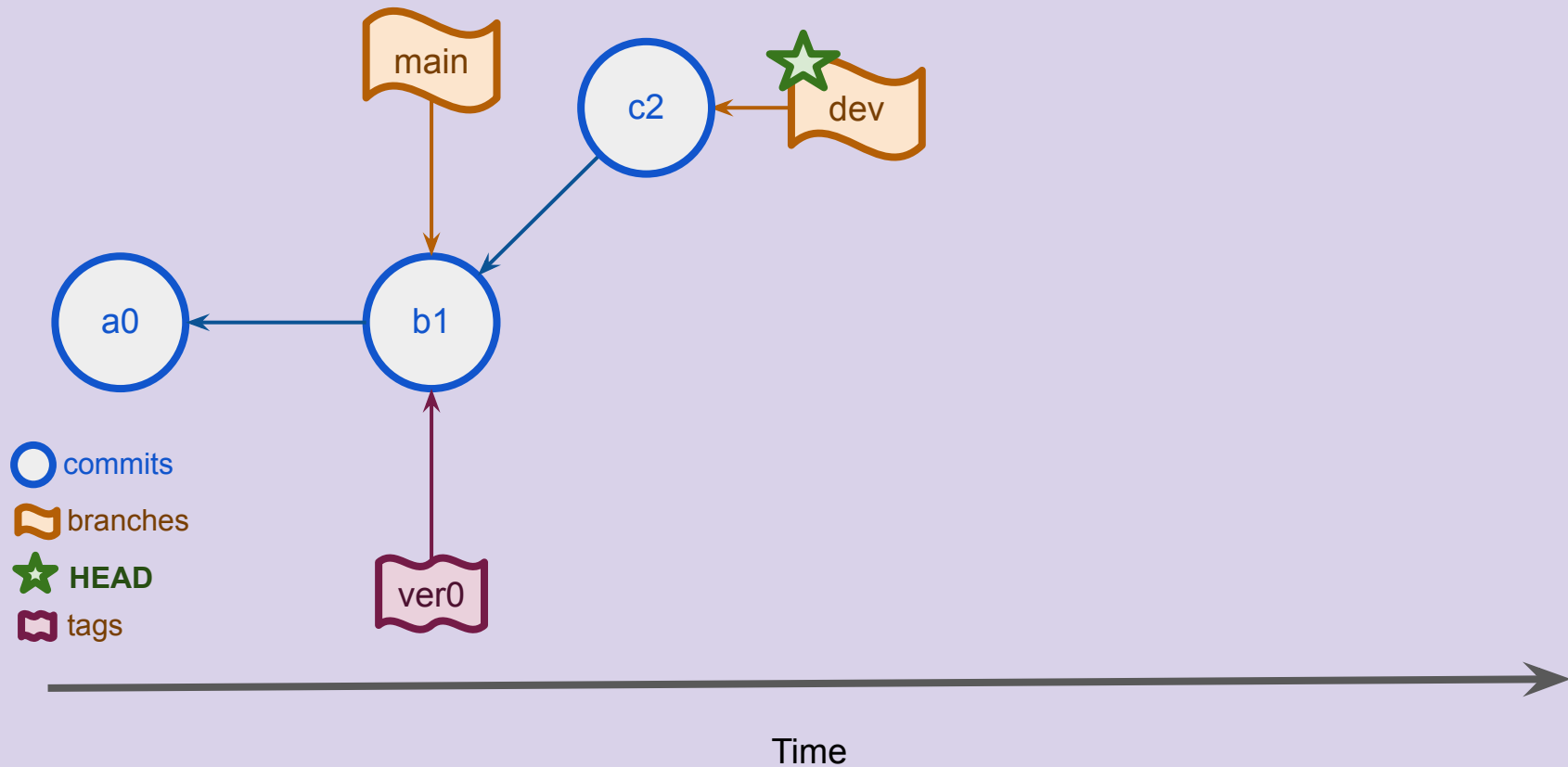
Moves head to a different branch and retains local changes

Flag -c creates a new branch “git switch -c new_branch”

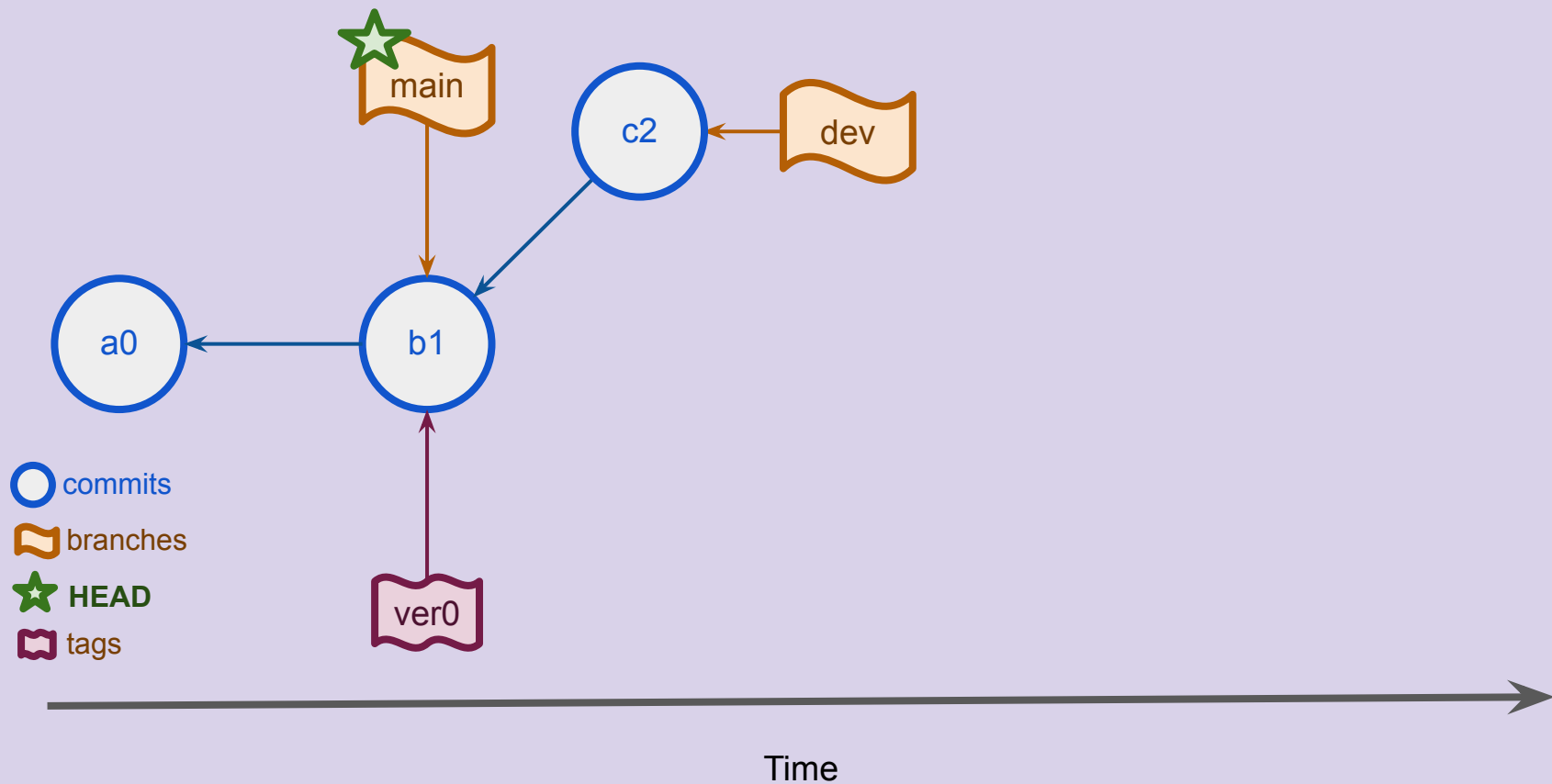
Generally supposed to replace older git checkout new_branch

***will not overwrite local files** by default if they have untracked changes that diverge from the files on *branch_name*

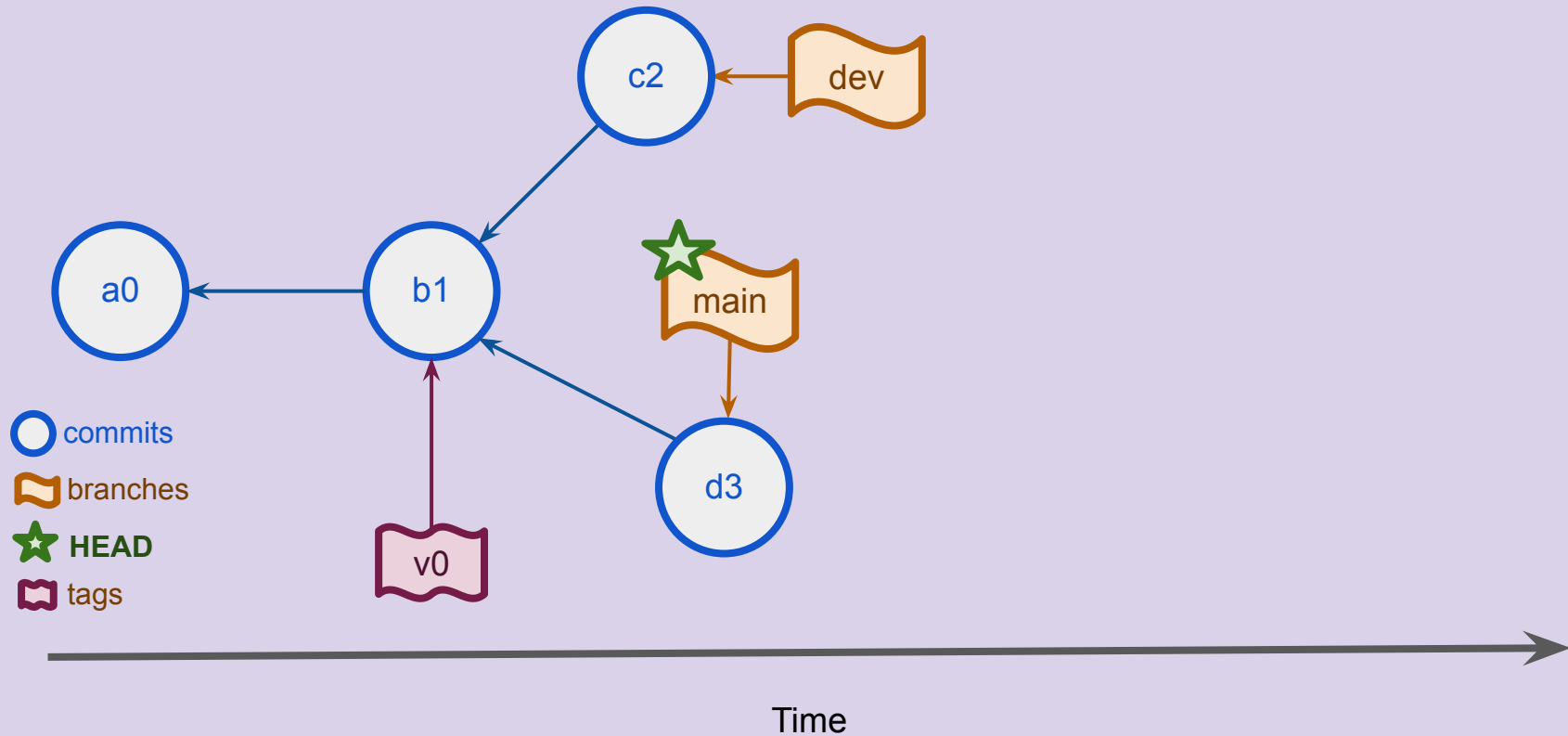
Merging



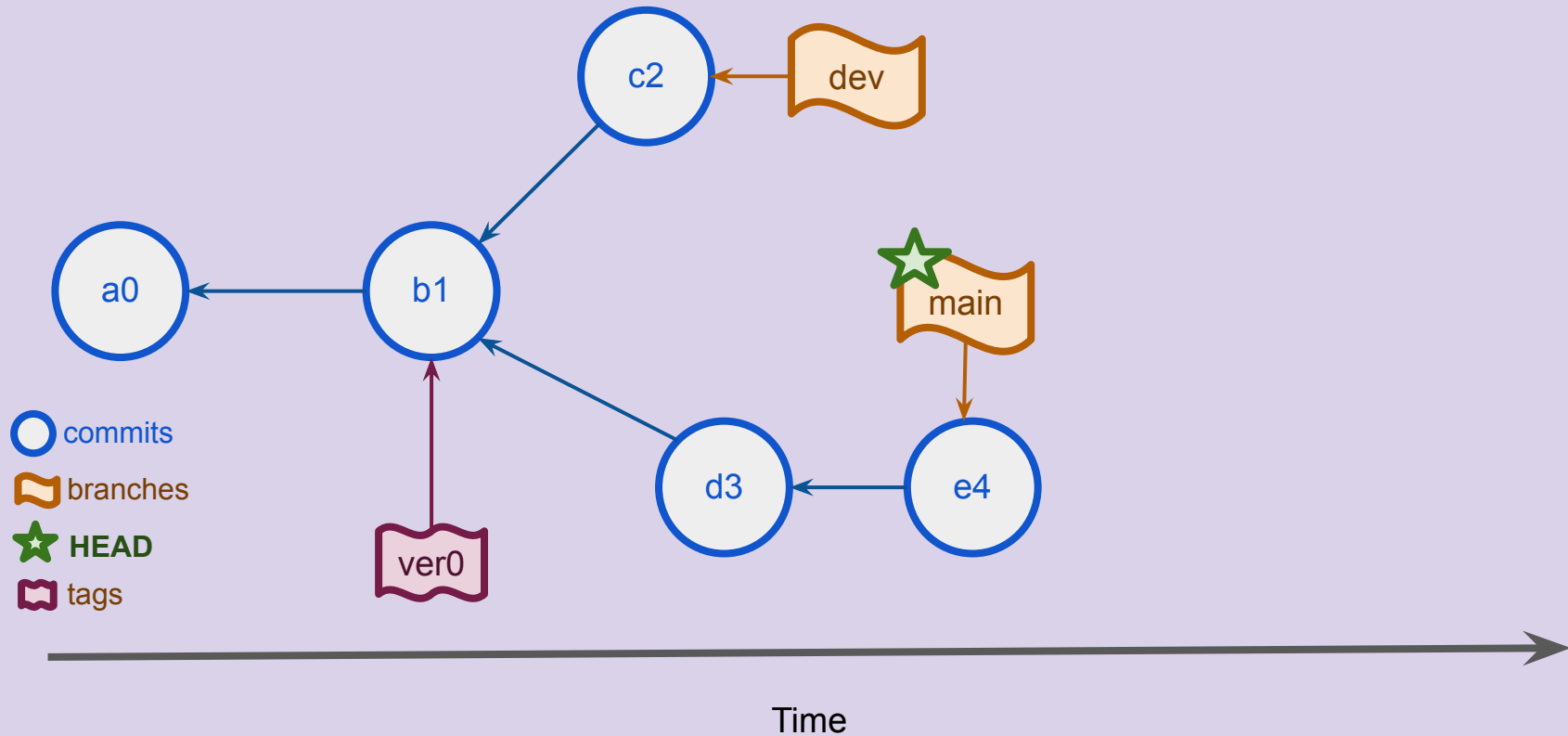
```
$ git commit
```



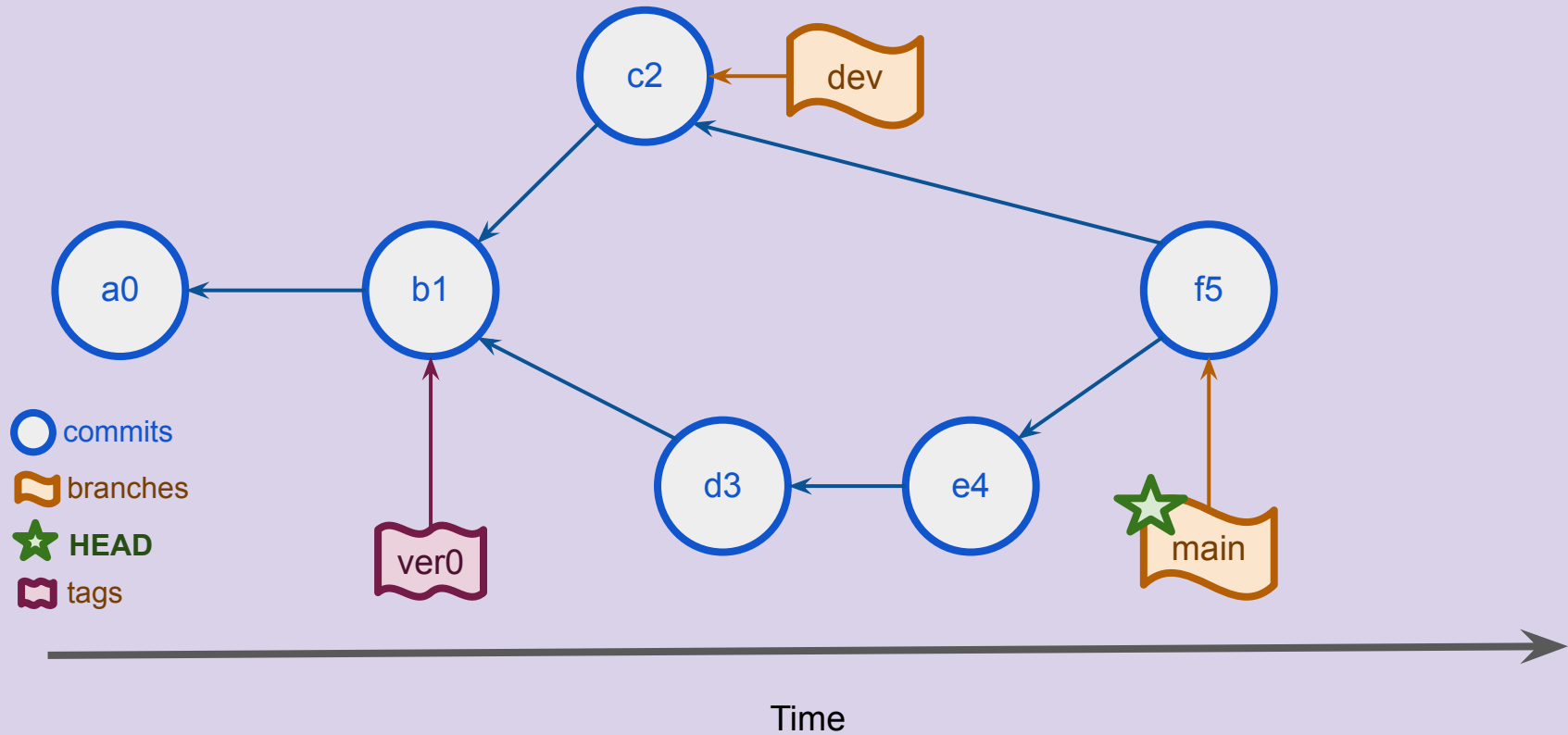
```
$ git switch main
```



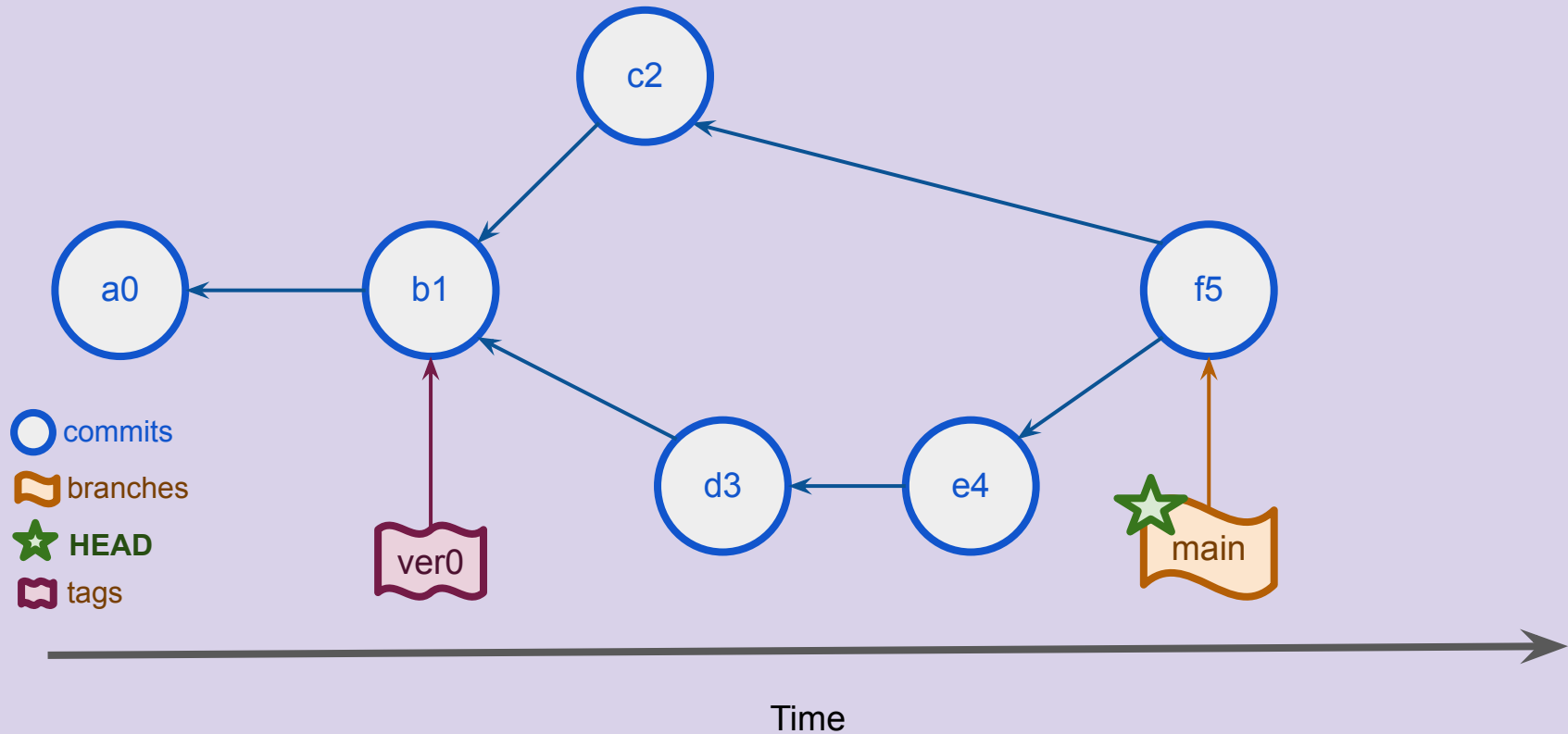
```
$ git commit
```



```
$ git commit
```



```
$ git merge dev
```



```
$ git branch --delete dev
```

git merge branch_name

Merges two branches

First switch to the branch you want to merge your new branch to, e.g, main

Merges “branch_name” to the branch head is currently on, e.g., main

Requires a commit message, typically through a text editor, for the merge to finalize

Important: if you edit the same file in both branches, you may have a merge conflict!

Lab Notebook Part 2

Reading Checkpoint #2